ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

VNU–UET

BÁO CÁO THỰC HÀNH MÔN HỌC

TƯƠNG TÁC NGƯỜI – ROBOT

BÀI THỰC HÀNH 05

MÔ PHỎNG TURTLEBOT 4
ĐỊNH VỊ AMCL, ĐIỀU HƯỚNG NAV2
VÀ NHẬN DIỆN VẬT THỂ BẰNG YOLO

Giảng viên hướng dẫn:	PGS. TS. Hoàng Văn Xiêm KS. Nguyễn Quốc Bảo
Sinh viên thực hiện:	Lê Anh Tuấn Bằng
Mã số sinh viên:	23020723
Nhánh mã nguồn:	`assignments_2`
Package chính:	`turtlebot4_lab05`

Hà Nội, 2026

Mục lục

1 Mục tiêu, phạm vi và môi trường triển khai **1**

1.1 Yêu cầu bài thực hành 1

1.2 Môi trường và khả năng tái lập 1

2 Cấu trúc chương trình và kiến trúc tổng thể **2**

2.1 Phân chia trách nhiệm trong mã nguồn 2

2.2 Hai luồng xử lý chạy đồng thời 3

3 Package, node và giao diện ROS 2 **4**

3.1 Các node/package theo từng chức năng 4

3.2 Lifecycle và điều kiện active 4

3.3 Các topic chính và ý nghĩa dữ liệu 5

3.4 Action và service trong quá trình điều hướng 6

4 Nạp lại bản đồ và định vị bằng AMCL **7**

4.1 Nguồn bản đồ và cách map server diễn giải ảnh 7

4.2 Bản đồ tĩnh và vật thể thêm vào simulator 7

4.3 AMCL: dự đoán, hiệu chỉnh và tái lấy mẫu 8

4.4 Chuỗi TF và pose khởi tạo 8

5 Cách Nav2 lập kế hoạch và điều khiển robot **9**

5.1 Từ mục tiêu tới vận tốc 9

5.2 Costmap, thông số và Behavior Tree 10

5.3 Phân biệt các lớp điều phối 10

6 Giải thích chương trình điều hướng tuần tự **11**

6.1 Waypoint và kiểm tra trước khi gửi goal 11

6.2 Readiness: vì sao chương trình có thể chờ? 11

6.3 Máy trạng thái, timeout và hủy goal 12

7 Pipeline nhận diện vật thể YOLO chạy song song **13**

7.1 Nhận ảnh, suy luận và xuất kết quả 13

7.2 Ngưỡng theo lớp và xác nhận theo thời gian 14

7.3 Dữ liệu đầu ra và giới hạn nhận diện 14

8 Bố trí vật thể trên bản đồ demo **15**

9 Quy trình chạy và trình bày demo 16

9.1 Chuẩn bị môi trường và tài nguyên trước khi quay 16

9.2 Bắt đầu quay khi nạp map và đặt initial pose 16

9.3 Chạy YOLO và chuỗi ba goal khi đang quay 17

9.4 Quan sát và ghi bằng chứng 17

10 Kết quả thực nghiệm và hình ảnh minh họa 18

10.1 Ảnh trích trực tiếp từ video demo đã cung cấp 18

10.2 Nạp map, localization và đường đi trên RViz2 19

10.3 Ba goal liên tiếp và quỹ đạo thực đo 20

10.4 Nhận diện thực tế và bằng chứng chạy đồng thời 21

11 Kiểm tra package và đánh giá kết quả 22

11.1 Kiểm tra mã nguồn ngày 20/10/2026 22

11.2 Cách đối chiếu một mission thành công 22

12 Thảo luận, kiến thức rút ra và hướng phát triển 23

12.1 Những điểm cần hiểu khi giải thích chương trình 23

12.2 Giới hạn và cách cải thiện có thể kiểm chứng 23

12.3 Kết luận 23

Tài liệu tham khảo và tài nguyên nộp bài 24

Danh sách hình vẽ

1 Kiến trúc tổng thể của hệ thống . . . 3
2 Quan hệ các frame map, odom và base link . . . 8
3 Pipeline lập kế hoạch và điều khiển Nav2 . . . 9
4 Máy trạng thái gửi goal tuần tự . . . 12
5 Pipeline xử lý ảnh và nhận diện YOLO . . . 13
6 Map, ba goal và tám vật thể hiện tại . . . 15
7 Video: RViz và YOLO hoạt động đồng thời . . . 18
8 Video: xe đạp và ghế được YOLO nhận diện . . . 18
9 Map, localization và đường đi trên RViz . . . 19
10 Quỹ đạo AMCL và ground truth của lượt baseline . . . 20
11 Kết quả nhận diện person, horse và chair . . . 21
12 Số liệu nhận diện khi robot di chuyển . . . 21

Danh sách bảng

1 Yêu cầu và tiêu chí quan sát được . . . 1
2 Các thành phần phần mềm của workspace . . . 1
3 Các tệp chính của package và vai trò . . . 2
4 Các thành phần tham gia pipeline . . . 4
5 Topic, kiểu message và hướng truyền dữ liệu . . . 5
6 Các action/service chính . . . 6
7 Thông số Nav2 đọc từ cấu hình chính thức đang cài . . . 10
8 Ba mục tiêu được đọc từ cấu hình waypoint . . . 11
9 Cấu hình nhận diện hiện tại . . . 14
10 Tọa độ và khoảng hở tới tường maze của scene hiện tại . . . 15
11 Các mốc phải xuất hiện trong video . . . 17
12 Kết quả baseline và sai số vị trí gần thời điểm action kết thúc . . . 20
13 Tóm tắt lượt sáu vật thể đã lưu log . . . 21
14 Phạm vi và kết quả kiểm tra trong lần rà soát hiện tại . . . 22

Danh mục đoạn mã

1 Các trường của map YAML dùng trong bài 05 . . . 7
2 Trích logic xử lý kết quả trong sequential_navigation.py . . . 12
3 Chuẩn bị workspace và tài nguyên trước lượt demo . . . 16
4 Terminal A và C: simulator, vật thể và undock . . . 16
5 Terminal B: nạp map cũ, AMCL và Nav2 . . . 16
6 Terminal D, E, F: nhận diện, observer và mission . . . 17
7 Lệnh tái lập kiểm tra không điều khiển robot . . . 22

1 Mục tiêu, phạm vi và môi trường triển khai

1.1 Yêu cầu bài thực hành

Bài thực hành tích hợp định vị trên bản đồ có sẵn, điều hướng tự động và nhận diện ảnh RGB trên TurtleBot 4 mô phỏng. Bản đồ đã xây dựng ở bài 04 được sử dụng lại; robot tự đi qua ba mục tiêu có thứ tự, trong khi cửa sổ camera hiển thị kết quả YOLO cập nhật theo thời gian. Hai luồng điều hướng và nhận diện chạy đồng thời qua ROS 2.

Bảng 1: Yêu cầu và tiêu chí quan sát được

Nội dung	Tiêu chí hoàn thành
Nạp map và localization	Map cũ xuất hiện trong RViz2; AMCL hoạt động; TF nối từ map tới robot; LiDAR phù hợp với tường trên map.
Điều hướng tuần tự	Code gửi Goal_1, Goal_2, Goal_3; mỗi action có kết quả SUCCEEDED trước khi gửi mục tiêu kế tiếp.
Nhận diện song song	Camera thật của simulator được xử lý thành bbox, lớp, confidence; có ảnh annotated trong lúc odometry cho thấy robot chuyển động.

1.2 Môi trường và khả năng tái lập

Bảng 2: Các thành phần phần mềm của workspace

Thành phần	Phiên bản/cấu hình
Hệ điều hành; ROS	Ubuntu 22.04; ROS 2 Humble; Python 3.10.12
Simulator; robot	Ignition Gazebo Fortress 6.18.0; world maze; TurtleBot 4 standard
Điều hướng	`turtlebot4_navigation`, Nav2, AMCL; tham số chính thức của bản Humble đang cài
Nhận diện hiện tại	Ultralytics 8.3.221; YOLO11s; ảnh đích 640; lọc sáu lớp COCO
Thư viện ảnh/tensor	PyTorch 2.5.1+cu121; torchvision 0.20.1+cu121; OpenCV 4.10.0.84; NumPy 1.26.4
GPU trong log kiểm chứng	NVIDIA GeForce RTX 3060 Laptop, 6144 MiB; CUDA runtime 12.1
Mạng ROS của bài	`ROS_DOMAIN_ID=64`; `ROS_LOCALHOST_ONLY=1`

Báo cáo đối chiếu mã nguồn nền `a90f949` và kiểm tra package ngày 20/10/2026. Số liệu điều hướng có timestamp ngày 15/10/2026 được ghi rõ là lịch sử. Cấu hình hiện tại có 8 vật thể, YOLO11s và bộ lọc mới; không sử dụng số liệu của phiên bản YOLO cũ để khẳng định độ chính xác của phiên bản mới.

Tài nguyên nộp bài: Mã nguồn GitHub – assignments_2; Video demo – Google Drive. Các liên kết dùng nhãn ngắn và có thể bấm trực tiếp trong PDF.

2 Cấu trúc chương trình và kiến trúc tổng thể

2.1 Phân chia trách nhiệm trong mã nguồn

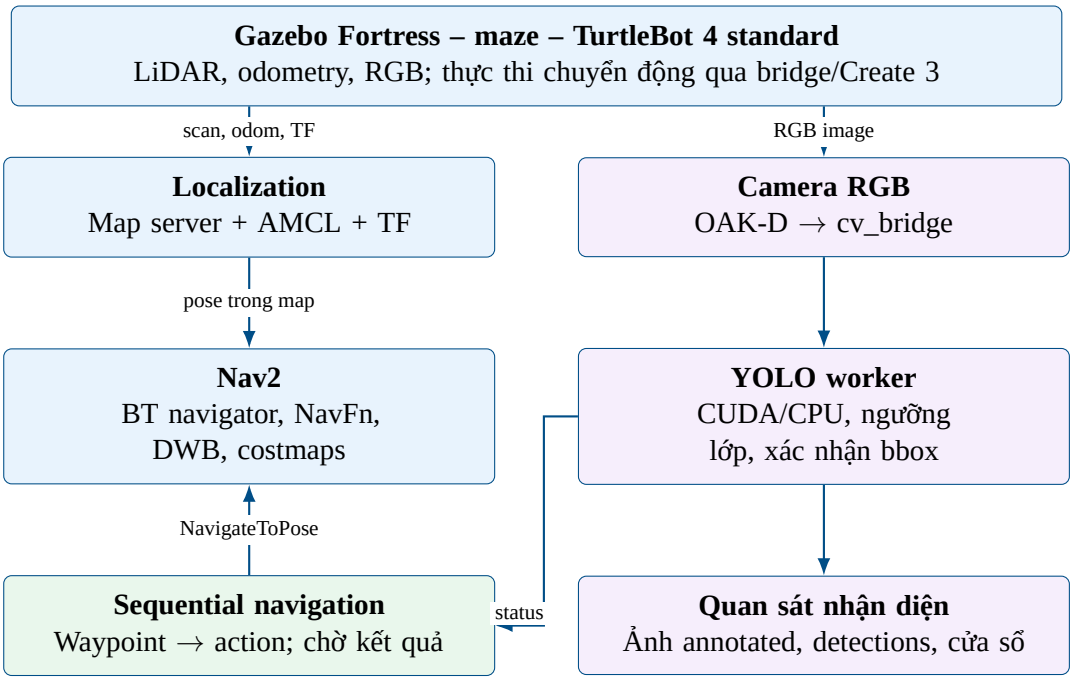
Bảng 3: Các tệp chính của package và vai trò

Tệp/thư mục	Trách nhiệm chính
`launch/bringup.launch.py`	Gọi launch localization/Nav2 của bài 04, RViz bài 05; có tùy chọn bật YOLO và mission. Mặc định chưa tự gửi goal.
`launch/yolo.launch.py`	Chạy riêng nhận diện; khai báo device, weights, ngưỡng và cửa sổ.
`sequential_navigation.py`	Action client bất đồng bộ, readiness, feedback, timeout, retry, cancel, dwell, log JSONL.
`configuration.py`	Kiểm tra waypoint, map hash, giá trị hữu hạn; đổi yaw sang quaternion; kiểm tra vùng trống.
`initialize_pose.py`	Khởi tạo AMCL trước mission từ pose đã xác thực hoặc nguồn mô phỏng được chọn tường minh.
`capture_waypoints.py`	Nhận điểm từ RViz, bổ sung yaw, kiểm tra map và lưu YAML.
`yolo_node.py`	Nhận ảnh mới nhất, suy luận trên worker, lọc bbox, xuất ảnh/JSON/status và cửa sổ OpenCV.
`detection_filter.py`	Xác nhận bbox cùng lớp qua các frame liền nhau bằng IoU; xóa trạng thái khi dữ liệu không hợp lệ/cũ.
`spawn_objects.py`, `scene_layout.py`	Chuẩn bị asset SDF; kiểm tra footprint với map và tường maze; thêm/thay model vào simulator.
`config/`, `maps/`	Waypoint, pose, YAML YOLO, vật thể, RViz; bản sao map từ bài 04.
`scripts/`, `validation/`	Môi trường Python, download, kiểm tra scene, observer; các snapshot kiểm chứng và ảnh thực tế.

Các module Python trong bảng nằm dưới `turtlebot4_lab05/`. Năm executable được khai báo trong `setup.py`: `sequential_navigation`, `yolo_node`, `initialize_pose`, `capture_waypoints` và `spawn_demo_objects`. Các script tiện ích được chạy trực tiếp bằng Python hoặc source trong shell.

Bài 05 tái sử dụng `turtlebot4_lab` để khởi động simulator/bridge và gọi localization/navigation chính thức. Cách này duy trì cùng world, robot, topic và map như bài trước, đồng thời tập trung phần mới vào mission và thị giác.

2.2 Hai luồng xử lý chạy đồng thời



Hình 1: Kiến trúc chức năng. Readiness của mission dùng status YOLO; kết quả bbox không đi vào bộ điều khiển Nav2. Luồng vận tốc do Nav2 phát về robot được mô tả ở Hình 3.

Trong mã thực tế, status YOLO được gửi tới node mission. Mũi tên status trong sơ đồ biểu diễn điều kiện cho phép bắt đầu điều hướng; thông tin nhận diện không tạo vật cản trong costmap. Các quan sát vật cản dùng LiDAR qua `/scan`.

Một callback camera chỉ lưu ảnh mới nhất và đánh thức worker. Suy luận nặng không chạy trong callback nhận ảnh. Node mission dùng action bất đồng bộ nên vẫn xử lý được feedback, heartbeat và tín hiệu hủy trong khi Nav2 chạy.

Chuyển động là vòng kín: cảm biến → AMCL/costmap → Nav2 → vận tốc → robot. Nhận diện là nhánh quan sát song song: camera → YOLO → hiển thị. Không có lệnh vận tốc được phát từ hai node Python của bài 05.

3 Package, node và giao diện ROS 2

3.1 Các node/package theo từng chức năng

Bảng 4: Các thành phần tham gia pipeline

Nhóm	Node/package tiêu biểu	Vai trò
Mô phỏng	`turtlebot4_ignition_bringup`, bridge, Create 3	World/robot; chuyển dữ liệu Ignition–ROS; thực thi vận tốc và undock.
Map	`nav2_map_server`: `map_server`	Đọc YAML và PGM, publish OccupancyGrid đã lưu.
Định vị	`nav2_amcl`: `amcl`	Kết hợp map, scan và odometry; ước lượng pose và TF map–odom.
Lập kế hoạch	`planner_server`, `nav2_navfn_planner`	Tính đường toàn cục trên global costmap.
Điều khiển	`controller_server`, `dwb_core`	Chọn vận tốc bám đường và tránh vật cản cục bộ.
Điều phối Nav2	`bt_navigator`, `behavior_server`	Thực thi Behavior Tree; replan, follow path, recovery.
Costmap/vận tốc	`nav2_costmap_2d`, `velocity_smoother`	Cập nhật lớp static/obstacle/inflation; làm mượt vận tốc.
Lifecycle	`lifecycle_manager_*`	Configure/activate các node quản lý lifecycle; mission đọc GetState.
Mission bài 05	`lab05_sequential_navigation`	Gửi ba NavigateToPose; ghi kết quả từng UUID.
Thị giác bài 05	`lab05_yolo`, `cv_bridge`, Ultralytics	Chuyển RGB sang BGR; inference; xuất ảnh annotated và JSON.
Giao diện	`rviz2`, OpenCV HighGUI	Map, scan, TF, path, costmap và cửa sổ camera.

YOLO/PyTorch là thư viện Python được node sử dụng, không phải các node ROS độc lập. `nav2_simple_commander` có trong dependency, nhưng mission này trực tiếp tạo `rclpy.action.ActionClient`; không dùng `BasicNavigator` hay action FollowWaypoints để chạy chuỗi mục tiêu.

3.2 Lifecycle và điều kiện active

Mission gọi `/<node>/get_state` kiểu `lifecycle_msgs/srv/GetState`. Các node được kiểm tra gồm AMCL, map server, planner, controller, BT navigator, behavior server, velocity smoother và hai costmap. State id 3 là active. Tồn tại process hoặc xuất hiện trong `ros2nodelist` chưa đủ để khẳng định pipeline đã sẵn sàng.

3.3 Các topic chính và ý nghĩa dữ liệu

Bảng 5: Topic, kiểu message và hướng truyền dữ liệu

Topic	Kiểu message	Nguồn → nơi sử dụng
`/map`	`nav_msgs/msg/OccupancyGrid`	Map server → AMCL, static layer, RViz, mission.
`/scan`	`sensor_msgs/msg/LaserScan`	LiDAR/bridge → AMCL, costmap, RViz, readiness.
`/odom`	`nav_msgs/msg/Odometry`	Odometry Create 3 → Nav2, observer.
`/tf`, `/tf_static`	`tf2_msgs/msg/TFMessage`	AMCL, robot publishers → biến đổi tọa độ.
`/clock`	`rosgraph_msgs/msg/Clock`	Simulator → ROS time của các node.
`/initialpose`	`geometry_msgs/msg/PoseWithCovarianceStamped`	RViz/helper → khởi tạo phân bố AMCL.
`/amcl_pose`	`geometry_msgs/msg/PoseWithCovarianceStamped`	AMCL → RViz, mission, observer.
`/plan`, `/local_plan`	`nav_msgs/msg/Path`	Planner/DWB → RViz và quan sát đường đi.
`/cmd_vel`	`geometry_msgs/msg/Twist`	Nav2/smoother → bridge/Create 3.
`/oakd/rgb/preview/image_raw`	`sensor_msgs/msg/Image`	Camera RGB → YOLO, observer.
`/oakd/rgb/preview/camera_info`	`sensor_msgs/msg/CameraInfo`	Thông tin nội tại; pipeline bbox 2D không cần dùng trực tiếp.
`/yolo/annotated_image`	`sensor_msgs/msg/Image`	YOLO → RViz/observer; giữ header ảnh vào.
`/yolo/detections`	`std_msgs/msg/String` (JSON)	BBox, class, confidence, stamp và device.
`/yolo/status`	`std_msgs/msg/String` (JSON)	Readiness, freshness, số frame, thiết bị model.
`/lab05/navigation_status`	`std_msgs/msg/String` (JSON)	Trạng thái mission, UUID, kết quả, hoàn thành.
`/clicked_point`	`geometry_msgs/msg/PointStamped`	Publish Point → capture waypoint; không có yaw.
`/sim_ground_truth_pose`	`nav_msgs/msg/Odometry`	Đối chiếu mô phỏng; tùy chọn khởi tạo trước mission.
`/hazard_detection`	`irobot_create_msgs/msg/HazardDetectionVector`	Observer ghi các cảnh báo từ Create 3.

QoS: map, AMCL và status được subscribe bằng Transient Local để nhận trạng thái gần nhất khi node khởi động muộn. RGB và ảnh annotated dùng Best Effort, depth 1; mục tiêu là xử lý frame gần hiện tại, chấp nhận bỏ frame khi tải cao. QoS không phù hợp có thể khiến topic tồn tại nhưng subscriber không nhận dữ liệu.

3.4 Action và service trong quá trình điều hướng

Bảng 6: Các action/service chính

Tên	Kiểu	Vai trò
`/navigate_to_pose`	`nav2_msgs/action/NavigateToPose`	Mission gửi một PoseStamped; BT navigator nhận và điều phối.
`/compute_path_to_pose`	`nav2_msgs/action/ComputePathToPose`	BT gọi planner để tính global path.
`/follow_path`	`nav2_msgs/action/FollowPath`	BT yêu cầu controller bám path.
`/spin`, `/backup`, `/wait`	Action của `nav2_msgs`	Recovery tùy nhánh của Behavior Tree.
`/undock`	`irobot_create_msgs/action/Undock`	Đưa robot rời dock trước khi demo.
`/request_nomotion_update`	`std_srvs/srv/Empty`	Yêu cầu AMCL cập nhật khi robot đứng yên.
`/<node>/get_state`	`lifecycle_msgs/srv/GetState`	Mission kiểm tra state active.

Vì sao dùng action? Một lượt di chuyển có thể kéo dài hàng chục giây, cần phản hồi tiến độ và có thể hủy. Action cung cấp goal acceptance, UUID, feedback, result và cancel. Topic phù hợp luồng cảm biến liên tục; service phù hợp yêu cầu ngắn như đọc lifecycle hoặc no-motion update.

`NavigateToPose.Goal` gồm `pose` và `behavior_tree`. Code gán pose; chuỗi behavior tree để trống nên Nav2 dùng cây mặc định. Feedback của bản Humble có current pose, navigation time, estimated remaining time, recoveries và distance remaining. Code ghi khoảng cách còn lại và số recovery mỗi khoảng hai giây.

Kết quả thành công đọc ở đâu? Trường result của action Humble chỉ chứa Empty; status kết thúc nằm trong wrapper trả về bởi `get_result_async()`. Code đọc `response.status`: 4 = SUCCEEDED, 5 = CANCELED, 6 = ABORTED. Accepted chỉ cho biết server nhận yêu cầu, chưa chứng minh robot đã tới đích.

Namespace của DWB: RViz trong cấu hình này đọc `/local_plan`; khi đổi namespace/plugin cần kiểm tra lại bằng `ros2topiclist`. Topic path để hiển thị không thay thế trạng thái action làm tiêu chí hoàn thành mission.

4 Nạp lại bản đồ và định vị bằng AMCL

4.1 Nguồn bản đồ và cách map server diễn giải ảnh

`maps/map.pgm` là bản sao ảnh map đã xây dựng ở bài 04. Kiểm tra SHA256 xác nhận ảnh bài 04 và bài 05 giống nhau. YAML của bản sao quy định resolution $r = 0.05$ m/pixel, origin $(-10.1, -10.2, 0)$, occupied threshold 0.65 và free threshold 0.196. Ảnh map gốc không bị sửa để đặt thêm đồ vật.

Đoạn mã 1: Các trường của map YAML dùng trong bài 05

```
image: map.pgm
mode: trinary
resolution: 0.05
origin: [-10.1, -10.2, 0.0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.196
```

Với ảnh xám $I \in [0, 255]$ và negate bằng 0, xác suất chiếm chỗ được suy ra từ $p = 1 - I/255$. Pixel trắng là vùng trống, đen là vật cản; vùng nằm giữa hai ngưỡng là chưa biết. Ví dụ pixel 205 cho $p \approx 0.196078 > 0.196$ nên giữ unknown, tránh coi vùng chưa quét là sàn trống.

Map server đọc metadata và ảnh, phát `/map` kiểu OccupancyGrid. Thông tin origin/resolution xác định phép chuyển tọa độ mét sang ô lưới. Với origin yaw bằng 0:

$$u = (x - x_0)/r, \qquad v = (y - y_0)/r. \tag{1}$$

Khi truy cập ảnh PGM để vẽ, trục hàng ảnh đi xuống nên phải đảo hướng so với y của map. Hàm `point_is_free()` làm việc trên OccupancyGrid và còn xử lý origin quay, biên map, unknown và ô có giá trị chiếm chỗ từ 50 trở lên.

4.2 Bản đồ tĩnh và vật thể thêm vào simulator

Các vật thể demo không được vẽ đè vào PGM để tạo kết quả localization giả. Static layer tiếp tục dùng bản đồ tuần trước. LiDAR quan sát vật thể mới và cập nhật obstacle/voxel layer của costmap. Inflation layer mở rộng chi phí xung quanh vật cản để planner/controller dành khoảng hở cho robot.

Cùng một ảnh PGM chưa đủ bảo đảm cùng hệ tọa độ nếu đổi origin, resolution hoặc world. Các YAML waypoint/scene chứa hash ảnh; readiness còn kiểm tra điểm goal trên OccupancyGrid đang nhận. Scene chỉ được dùng với maze và map/world alignment đã xác thực.

4.3 AMCL: dự đoán, hiệu chỉnh và tái lấy mẫu

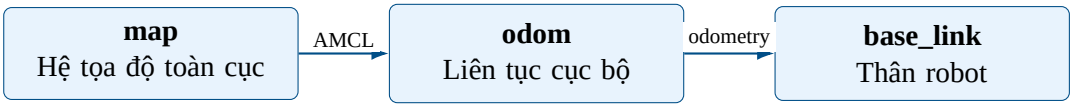
AMCL biểu diễn phân bố pose robot bằng tập hạt $s_t^{(i)} = (x, y, \theta)$. Khi có chuyển động odometry u_t, hạt được dự đoán theo mô hình vi sai; khi có scan z_t, likelihood của từng hạt được tính bằng độ phù hợp giữa điểm LiDAR và tường trên map. Chu trình ở mức khái niệm:

$$s_t^{(i)} \sim p(s_t \mid s_{t-1}^{(i)}, u_t), \tag{2}$$

$$\widetilde{w}_t^{(i)} \propto p(z_t \mid s_t^{(i)}, m), \qquad w_t^{(i)} = \frac{\widetilde{w}_t^{(i)}}{\sum_j \widetilde{w}_t^{(j)}}. \tag{3}$$

Sau hiệu chỉnh, tái lấy mẫu tập trung hạt vào các pose có trọng số cao. Cấu hình đang dùng `likelihood_field`, `DifferentialMotionModel`, 500–2000 hạt, tối đa 60 beam và `resample_interval=1`. Các giá trị này đọc từ `turtlebot4_navigation/config/localization.yaml` của bản Humble cài trên máy.

4.4 Chuỗi TF và pose khởi tạo



Hình 2: Chuỗi TF dùng để xác định pose robot trong map. Các frame sensor được nối tiếp qua TF của robot.

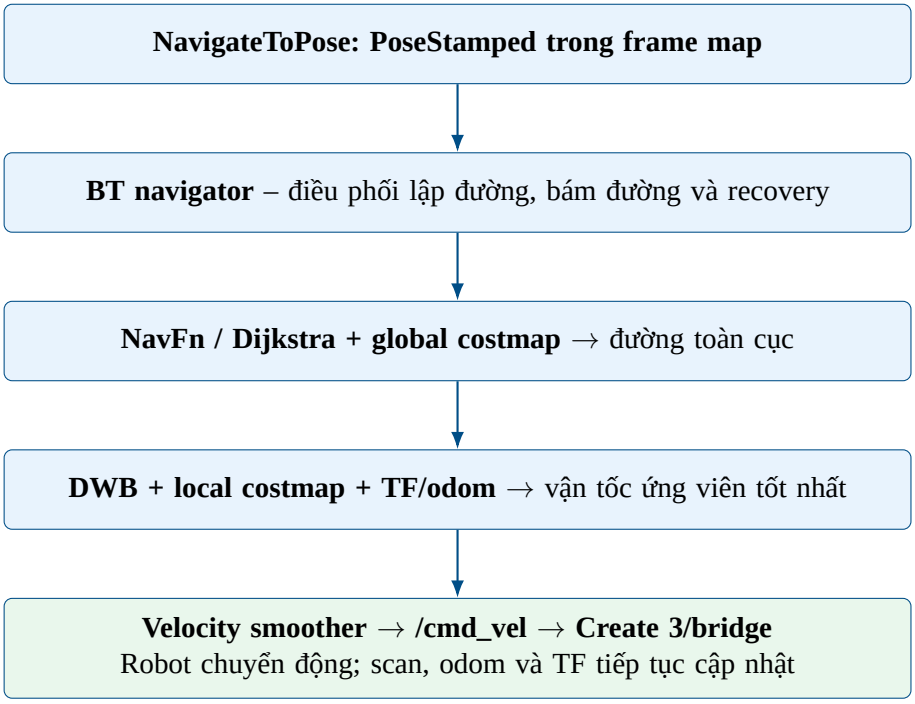
Odometry tích lũy chuyển động mượt nhưng trôi dần. AMCL ước lượng hiệu chỉnh map–odom dựa trên LiDAR; kết hợp hai biến đổi cho pose map–base_link. Không nên có hai node cùng phát một biến đổi map–odom; vì vậy không chạy SLAM đồng thời với AMCL cho lượt demo này.

Trên RViz, chọn **2D Pose Estimate**, bấm vị trí robot và kéo mũi tên theo hướng đầu robot. Message `/initialpose` chứa pose và covariance để khởi tạo phân bố hạt. Sau đó kiểm tra scan khớp mép tường, localization active và TF hợp lệ. Việc thấy một mũi tên pose chưa tự chứng minh AMCL đúng.

Helper `initialize_pose` có chế độ `source:=sim_ground_truth`, dùng pose mô phỏng một lần trước mission khi hash map và alignment hợp lệ. Nếu dùng cách này phải nêu rõ trong video. Sau khởi tạo, định vị vẫn do AMCL thực hiện; ground truth chỉ phục vụ đối chiếu, không liên tục ghi đè AMCL khi robot chạy.

5 Cách Nav2 lập kế hoạch và điều khiển robot

5.1 Từ mục tiêu tới vận tốc



Hình 3: Chuỗi xử lý điều hướng của cấu hình đang dùng. Costmap và cảm biến được cập nhật trong suốt quá trình.

Global planner. Plugin `nav2_navfn_planner/NavfnPlanner`, id `GridBased`, có `use_astar=false`: sử dụng Dijkstra trên lưới costmap, lan truyền chi phí và truy hồi đường tới goal. Đây là cấu hình NavFn, không phải Smac hay thuật toán A* tự viết. Planner có tolerance 0.5 m và `allow_unknown=true`; dù vậy client vẫn yêu cầu vị trí goal nằm trong vùng known-free có margin.

Local controller. Plugin `dwb_core::DWBLocalPlanner` lấy mẫu vận tốc, mô phỏng quỹ đạo ngắn, loại quỹ đạo không hợp lệ và chấm điểm các ứng viên. Các critic gồm RotateToGoal, Oscillation, BaseObstacle, GoalAlign, PathAlign, PathDist, GoalDist. Ở mức khái niệm, lựa chọn tương ứng:

$$(v^*, \omega^*) = \arg \min_{(v,\omega)\in\mathcal{V}_{\text{valid}}} \sum_k \alpha_k C_k(v, \omega). \tag{4}$$

Công thức giải thích cơ chế chấm điểm, không thay thế mã DWB. Robot đi theo từng lệnh vận tốc ngắn và tính lại liên tục khi cảm biến/pose thay đổi.

5.2 Costmap, thông số và Behavior Tree

Bảng 7: Thông số Nav2 đọc từ cấu hình chính thức đang cài

Thành phần	Thông số và tác động
Global costmap	Frame map; resolution 0.06 m; static, obstacle, inflation layer; cập nhật 1 Hz; theo dõi unknown.
Local costmap	Frame odom; rolling window 3×3 m; resolution 0.06 m; static, voxel, inflation; cập nhật 5 Hz.
Inflation	Robot radius 0.175 m; inflation radius 0.45 m; cost scaling factor 4.0.
Controller	20 Hz; $v_x \leq 0.26$ m/s; $\lvert\omega\rvert \leq 1.0$ rad/s; DWB dự báo 1.7 s.
Goal checker	SimpleGoalChecker; xy tolerance 0.25 m; yaw tolerance 0.25 rad; stateful true.
Progress checker	Phải dịch chuyển 0.5 m trong thời gian cho phép 10 s theo cấu hình SimpleProgressChecker.
Velocity smoother	20 Hz; giới hạn vận tốc tuyến tính $[-0.26, 0.26]$ m/s; góc $[-1, 1]$ rad/s.

Các giá trị trên là cấu hình được launch kế thừa, không phải số đo tần số thực tế. Nếu mô phỏng có real-time factor thấp, thời gian thực để robot đi hết đường tăng lên dù thông số vận tốc trong thời gian mô phỏng giữ nguyên.

Behavior Tree mặc định `navigate_to_pose_w_replanning_and_recovery.xml` lập lại đường toàn cục ở nhánh RateController 1 Hz, đồng thời chạy FollowPath. Khi planner/controller lỗi, nhánh tương ứng có thể xóa costmap; recovery tổng thể lần lượt có clear costmap, spin, wait, backup. Không phải lúc nào cũng chạy tất cả recovery: chỉ nhánh được kích hoạt mới thực thi.

5.3 Phân biệt các lớp điều phối

BT quản lý cách đi đến *một* goal. Node `sequential_navigation` quản lý *danh sách* ba goal: gửi một action, chờ result, dwell rồi chuyển mục tiêu. Retry của node bài 05 và retry/recovery nội bộ BT là hai mức khác nhau; không cộng số lượng chúng để suy ra chính xác số lần robot quay/lùi.

Kết quả SUCCEEDED được Nav2 đánh giá trên pose ước lượng và goal checker. Nó không có nghĩa tọa độ ground truth trùng hoàn toàn với waypoint. Sai số đo sau action cũng chịu độ lệch thời điểm lấy mẫu của observer; báo cáo tách rõ hai nguồn pose khi đưa số liệu.

6 Giải thích chương trình điều hướng tuần tự

6.1 Waypoint và kiểm tra trước khi gửi goal

Bảng 8: Ba mục tiêu được đọc từ cấu hình waypoint

Tên	x (m)	y (m)	yaw (rad)
Goal_1	-1.112140	0.315419	3.128185
Goal_2	3.174917	-3.194671	3.128185
Goal_3	3.615194	5.427635	0.258135

Tọa độ có nguồn từ lượt NavigateToPose thành công của bài 04, được lưu trong `waypoints.yaml` cùng source UUID. Goal_3 sử dụng pose AMCL cuối đã đo, thay endpoint cũ quá sát vùng unknown. `load_waypoints()` bắt buộc validated, frame map, ít nhất ba goal, tên không trùng, source không rỗng, giá trị hữu hạn và hash ảnh map phù hợp.

Yaw không được truyền như một trường trực tiếp của Pose. Với chuyển động trên mặt phẳng, quaternion là:

$$(q_x, q_y, q_z, q_w) = \left(0, 0, \sin\frac{\theta}{2}, \cos\frac{\theta}{2}\right). \tag{5}$$

`capture_waypoints` nhận `/clicked_point`, vốn chỉ có vị trí x, y, z, nên lấy yaw từ AMCL hoặc tham số được chọn riêng. Publish Point trên RViz không chứa hướng mũi robot.

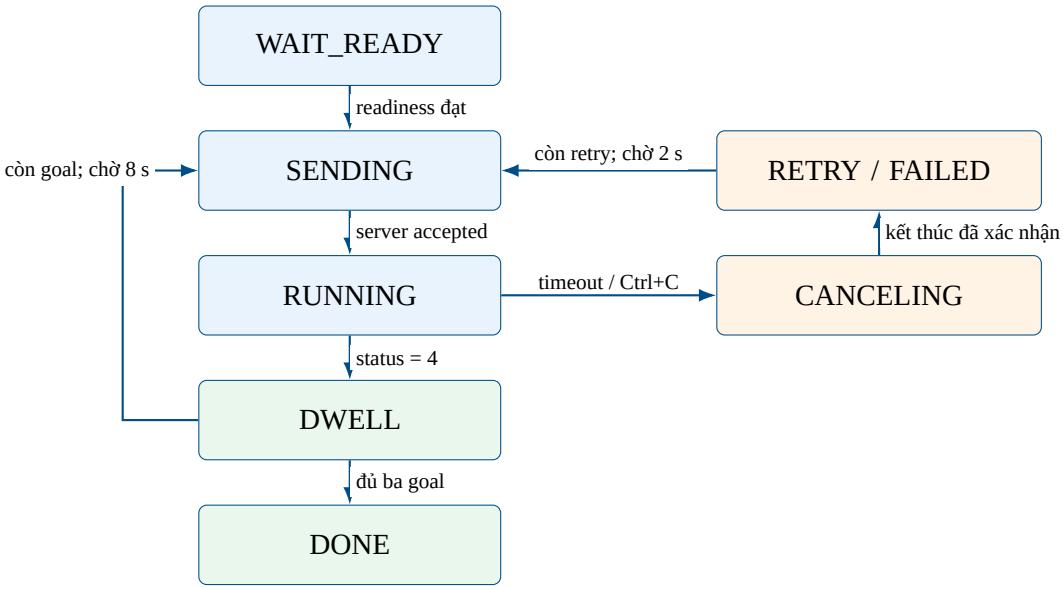
6.2 Readiness: vì sao chương trình có thể chờ?

Trước goal đầu và giữa các goal, client kiểm tra: người dùng xác nhận localization; clock đang tiến và scan còn mới; đã nhận map và AMCL; covariance vị trí không quá 0.3, yaw không quá 0.2; TF map–base_link không cũ quá 2 s; các lifecycle node active; action server có mặt; không có SLAM/teleop đang chạy; YOLO có inference thành công gần đây; toàn bộ vùng tròn quanh mỗi goal nằm trong known-free với margin 0.25 m.

Các giá trị covariance là phương sai (m^2, rad^2), không phải sai số mét/radian. Readiness không chứng minh localization đúng tuyệt đối: cần đối chiếu scan/map trên RViz trước khi đặt `localization_confirmed:=true`.

YAML cấu hình navigation đặt dwell 8 s; default trong constructor là 6 s. Khi chạy đúng lệnh với `--params-file config/navigation.yaml`, giá trị có hiệu lực là 8 s. Không nên bỏ params-file rồi suy ra hành vi giống kịch bản quay.

6.3 Máy trạng thái, timeout và hủy goal



Hình 4: Luồng trạng thái rút gọn của client. Khi không xác nhận được hủy trong giới hạn 8 s, node dừng với lỗi và không gửi retry.

Đoạn mã 2: Trích logic xử lý kết quả trong sequential_navigation.py

```
elif status == 4:
    self.index += 1
    self.attempt = 0
    self.state = 'DWELL'
    self.next_at = time.monotonic()+self.params['dwell_seconds']
    self.log('goal_succeeded', completed=self.index,
             dwell_seconds=self.params['dwell_seconds'])
else:
    self.failed(f'Action terminal status {status}; only 4 is success')
```

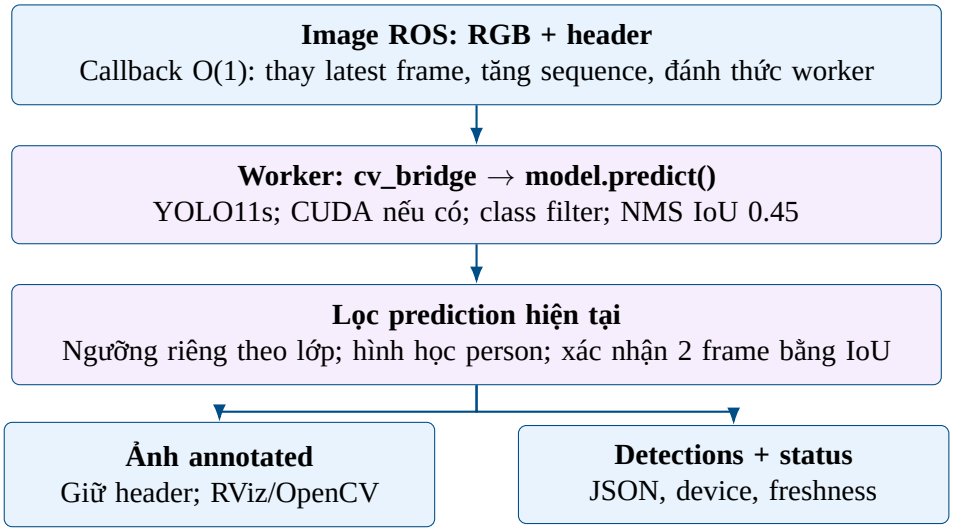
Readiness timeout 180 s; mỗi goal tối đa 300 s; tối đa một retry, chờ 2 s. Timeout dùng steady clock/wall monotonic để vẫn hết hạn khi simulator pause; header pose vẫn dùng ROS simulation time. Khi Ctrl+C, client gửi cancel và đợi kết quả, trả exit code 130. Nếu mất ACK/result, node không được tự suy ra rằng server đã dừng; nó báo lỗi sau giới hạn chờ, không gửi một goal khác có thể chạy chồng.

YOLO mất freshness khi đang chạy được đánh dấu `yolo_interrupted`; việc này không tự biến YOLO thành bộ lái hay tự làm action SUCCEEDED. Muốn chứng minh đủ yêu cầu video cần cả completed=3 và nhận diện chạy đồng thời, không chỉ nhìn dòng mission_completed.

7 Pipeline nhận diện vật thể YOLO chạy song song

7.1 Nhận ảnh, suy luận và xuất kết quả

Camera OAK-D trong simulator phát ảnh `rgb8`, kích thước 320×240, frame `oakd_rgb_camera_optical_frame`. `cv_bridge` chuyển message sang mảng NumPy BGR8 để đưa vào Ultralytics. `image_size=640` là kích thước đích của bước chuẩn bị tensor/letterbox; không tạo thêm chi tiết thật so với ảnh gốc.



Hình 5: Pipeline nhận diện. Ảnh annotated và JSON được tạo từ cùng danh sách bbox đã qua lọc.

Worker chỉ lấy sequence mới; ảnh cũ bị thay thế thay vì xếp hàng vô hạn. Sau mỗi vòng, code đặt thời điểm tiếp theo bằng thời điểm hiện tại cộng $1/\text{inference_rate}$. Vì còn thời gian inference, tham số 3.0 là giới hạn nhịp, không bảo đảm chính xác 3 FPS đầu ra. Callback ROS và GUI tiếp tục được xử lý khi worker chạy.

Weights được tải trước vào cache và pin SHA256 trong manifest. Node không tự download giữa demo. Khi chọn GPU, code kiểm tra CUDA và chỉ số device; sau inference đọc device thực của tensor tham số model. `selected_device=cuda:0` khi mới tải model chưa đủ chứng minh inference đã chạy trên GPU.

Nếu CUDA không có, `cpu_fallback=true` cho phép chọn CPU. Nếu inference phát lỗi CUDA/bộ nhớ, code chuyển model sang CPU; ba lỗi liên tiếp ngoài nhánh fallback làm node báo fatal. Nhánh này có trong code, nhưng không được tính là đã kiểm thử inference CPU chỉ từ phép thử không có camera.

7.2 Ngưỡng theo lớp và xác nhận theo thời gian

Bảng 9: Cấu hình nhận diện hiện tại

Tham số	Giá trị/cách áp dụng
Weights, device	`yolo11s.pt`; yêu cầu device 0; có CPU fallback
Danh sách lớp	person, bicycle, chair, potted plant, horse, fire hydrant
Ngưỡng chung	0.30 cho các lớp không có ngưỡng riêng
Ngưỡng person	0.55; chiều cao bbox ít nhất 35 pixel; $1.1 \leq h/w \leq 4.5$
Ngưỡng potted plant	0.15; tăng độ nhạy nhưng có thể tăng false positive
NMS IoU	0.45; xử lý bbox trùng trong cùng một ảnh
Xác nhận liên frame	2 frame; cùng class; bbox IoU ít nhất 0.25; timeout 1.5 s thực
Camera timeout	4 s; status phân biệt waiting_camera, warming_up, ready, error

Do lớp cây cần ngưỡng 0.15, lời gọi `model.predict()` dùng min của ngưỡng chung và cây. Sau đó code áp lại ngưỡng theo từng lớp. Nếu chỉ truyền conf=0.30 cho model thì bbox cây có confidence 0.20 đã bị loại trước khi đến bộ lọc riêng.

Với hai bbox A, B, độ chồng lấp:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}. \tag{6}$$

`DetectionConfirmation` ghép bbox cùng class với bbox frame trước có IoU lớn nhất, ưu tiên confidence cao và không cho một bbox trước xác nhận hai vật thể mới. Chỉ chỉ số prediction *hiện tại* đã đủ số lần khớp được trả về. Mất detection, đổi class, nhảy vị trí, stamp trùng/lùi hoặc quá timeout sẽ phá chuỗi xác nhận. Đây là lọc ngắn hạn, không phải hệ tracking có ID bền vững.

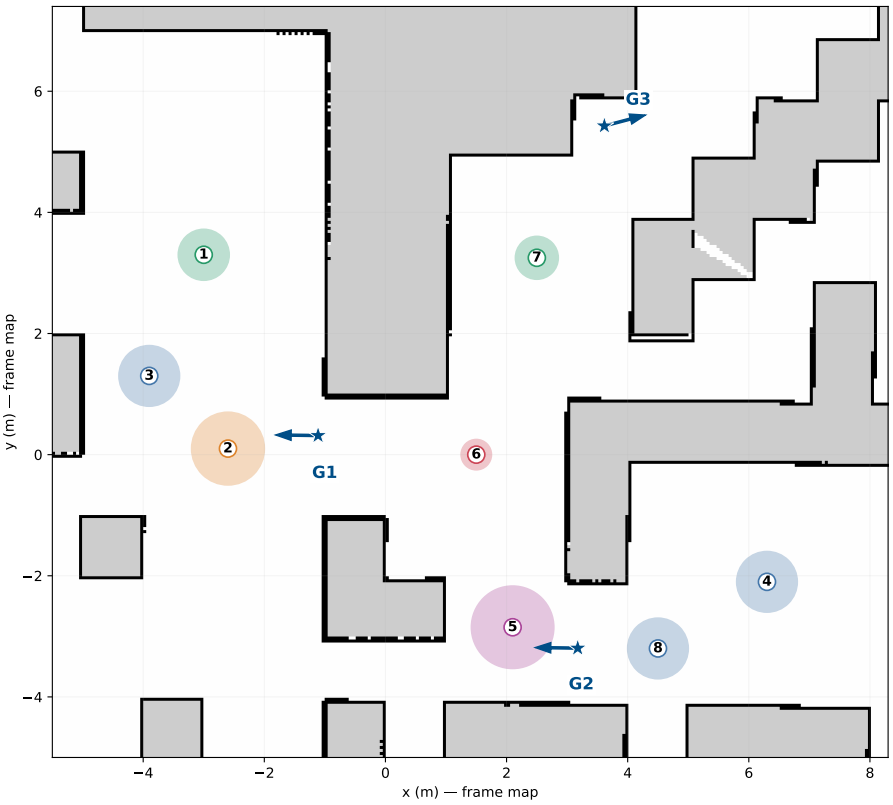
Hai giá trị IoU có chức năng khác nhau: NMS IoU so sánh các hộp trong cùng frame; confirmation IoU so sánh hộp qua hai frame. Tăng số frame xác nhận có thể giảm nhãn nhấp nháy nhưng tăng độ trễ và bỏ sót khi robot quay nhanh.

7.3 Dữ liệu đầu ra và giới hạn nhận diện

JSON detections chứa stamp, frame id, class id, label, confidence và xyxy theo pixel ảnh gốc. Ảnh vẽ bằng cùng tập index, copy nguyên header đầu vào; không dùng ảnh trang trí hay nhãn gán theo tên model Gazebo. Đóng preview bằng q/Esc chỉ tắt cửa sổ, không dừng inference.

Bbox 2D không cung cấp vị trí vật thể trong frame map. Pipeline chưa dùng depth, chiếu ngược camera, semantic map hoặc YOLO để phát goal. Lọc hình học person có thể loại nhầm người ngồi/xa/bị che; cây sồi 3D không đồng nghĩa với ground truth lớp COCO potted plant. Confidence lớn cũng không phải accuracy hay xác suất đúng đã hiệu chuẩn.

8 Bố trí vật thể trên bản đồ demo



Hình 6: Map đã lưu, ba Goal và tám vật thể hiện tại. Sao xanh chỉ goal; mũi tên chỉ yaw; đĩa màu bao footprint vật thể. Đây là sơ đồ dựng từ cấu hình và map, không phải ảnh một lần chạy mới.

Bảng 10: Tọa độ và khoảng hở tới tường maze của scene hiện tại

Số	Vật thể	x (m)	y (m)	Bán kính (m)	Hở tường (m)
1	Người	-3.00	3.30	0.420	1.580
2	Ngựa	-2.60	0.10	0.600	1.180
3	Ghế 1	-3.90	1.30	0.500	0.600
4	Ghế 2	6.30	-2.10	0.500	1.400
5	Xe đạp	2.10	-2.85	0.680	0.420
6	Trụ cứu hỏa	1.50	0.00	0.250	0.868
7	Cây sồi	2.50	3.25	0.354	1.146
8	Ghế 3	4.50	-3.20	0.500	0.443

`check_scene.py` kiểm tra đĩa bao toàn thân với known-free map, kể cả unknown/biên ảnh, box tường SDF có transform và khoảng hở từng cặp. Ngưỡng yêu cầu là 0.35 m với tường/vùng không trống và 0.60 m giữa vật thể. Cả tám vị trí đạt kiểm tra ngày 20/10/2026; khoảng hở nhỏ nhất trên map khoảng 0.385 m và với tường thật khoảng 0.420 m, đều ở xe đạp.

SDF giữ visual/texture asset gốc, chuyển đường dẫn texture sang cache cục bộ, áp scale rồi thay collision mesh bằng box. Balô và biển STOP nằm trong danh sách retired. Kiểm tra footprint-/collision box không chứng minh mọi đỉnh mesh hiển thị đều nằm trong đĩa bao hoặc mọi quỹ đạo sau này đều đi được; cần quan sát visual và chạy lại scene mới khi đánh giá thực nghiệm.

9 Quy trình chạy và trình bày demo

9.1 Chuẩn bị môi trường và tài nguyên trước khi quay

Mỗi terminal cần source cùng workspace và môi trường Python để ROS, cv_bridge và torch tương thích. Môi trường ảo bật system-site-packages cho thư viện ROS; script bổ sung site-packages vì executable ROS có thể dùng shebang Python hệ thống. Giới hạn thread CPU giúp giảm cạnh tranh tài nguyên với simulator.

Đoạn mã 3: Chuẩn bị workspace và tài nguyên trước lượt demo

```
cd ~/turtlebot4_lab_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install --packages-select turtlebot4_lab05
source lab_env.bash
source src/turtlebot4_lab05/scripts/lab05_env.bash
python src/turtlebot4_lab05/scripts/download_resources.py \
  --cache "$TB4_LAB05_CACHE"
python src/turtlebot4_lab05/scripts/check_scene.py
```

Các lệnh tiếp theo chạy ở terminal riêng, đều bắt đầu bằng `cd~/turtlebot4_lab_ws`, source `lab_env.bash` và `lab05_env.bash` như trên. Không mở hai simulator cùng partition hoặc hai bộ Nav2 trùng namespace.

Đoạn mã 4: Terminal A và C: simulator, vật thể và undock

```
# Terminal A: keep running
ros2 launch turtlebot4_lab simulation.launch.py \
  model:=standard rviz:=false

# Terminal C: after simulator is ready
ros2 run turtlebot4_lab05 spawn_demo_objects
ros2 action send_goal /undock \
  irobot_create_msgs/action/Undock '{}' --feedback
```

Chờ undock SUCCEEDED. Với world mới chỉ spawn một lần. Nếu đã có scene cũ, dùng `spawn_demo_objects--ros-args-preplace_existing:=true` thay lệnh spawn thường, trước khi bắt đầu mission. Nó chỉ thay các tên model lab05 được cấu hình và retired; không sửa tường/nền/robot.

9.2 Bắt đầu quay khi nạp map và đặt initial pose

Bấm Record **trước** khi chạy Terminal B để thấy quá trình nạp lại bản đồ. Sắp xếp RViz, camera và terminal mission sao cho đọc được kết quả mà vẫn thấy robot di chuyển.

Đoạn mã 5: Terminal B: nạp map cũ, AMCL và Nav2

```
ros2 launch turtlebot4_lab05 bringup.launch.py \
  start_yolo:=false start_navigation:=false
```

RViz phải có Fixed Frame map, Map, LiDAR scan, AMCL pose và costmap. Thao tác 2D Pose Estimate như phần định vị; đối chiếu scan với tường. Nếu dùng helper ground truth trước mission, thuyết minh rõ nguồn initial pose. Chưa bật mission khi localization còn sai.

9.3 Chạy YOLO và chuỗi ba goal khi đang quay

Đoạn mã 6: Terminal D, E, F: nhận diện, observer và mission

```
# Terminal D: real camera + YOLO window
ros2 launch turtlebot4_lab05 yolo.launch.py \
  device:=0 show_window:=true

# Terminal E: passive observer (optional, before mission)
python src/turtlebot4_lab05/scripts/record_demo.py \
  --output-directory "$TB4_LAB_WS/log/video_evidence"

# Terminal F: only after localization has been checked
export LAB05_VIDEO_LOG="$TB4_LAB_WS/log/navigation_video.jsonl"
ros2 run turtlebot4_lab05 sequential_navigation --ros-args \
  --params-file src/turtlebot4_lab05/config/navigation.yaml \
  -p localization_confirmed:=true \
  -p log_file:="$LAB05_VIDEO_LOG"
```

Dùng thư mục/tên log mới cho mỗi lượt quay để tránh trộn nhiều mission. Khi chạy F, không teleop, không bấm Nav2 Goal và không cập nhật initial pose giữa đường. Code sẽ tự gửi Goal_1, dừng 8 s sau thành công, rồi Goal_2 và Goal_3; không cần bấm Enter ở mỗi goal.

Bảng 11: Các mốc phải xuất hiện trong video

Mốc	Nội dung cần nhìn thấy/giải thích
Nạp map	Lệnh bringup, map cũ trên RViz, localization active.
Đặt pose	2D Pose Estimate hoặc helper được nói rõ; scan khớp map.
Bắt đầu mission	readiness_passed, goal_accepted; đường trên RViz và camera đang cập nhật.
Chuyển goal tự động	goal_result status=4 của Goal_1 rồi Goal_2; code tự chuyển sau dwell.
Kết thúc	Goal_3 status=4; mission_completed completed=3; camera vẫn mở; robot dừng.

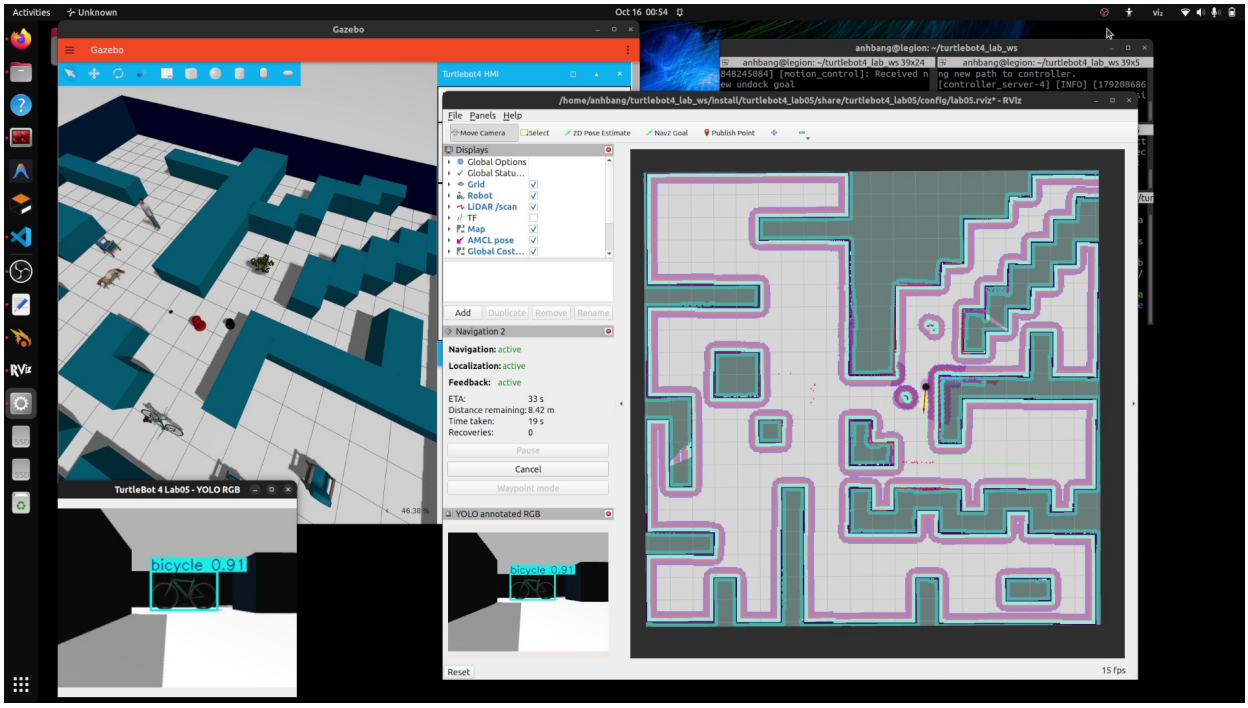
Giữ quay liên tục quá trình di chuyển và camera, rồi quay thêm 5–10 s bảng kết quả cuối. Không chỉ quay terminal in ba dòng SUCCEEDED vì chưa chứng minh robot đã thực hiện đường đi và YOLO hoạt động song song. Kịch bản chi tiết từng terminal nằm trong `VIDEO_DEMO.md`.

9.4 Quan sát và ghi bằng chứng

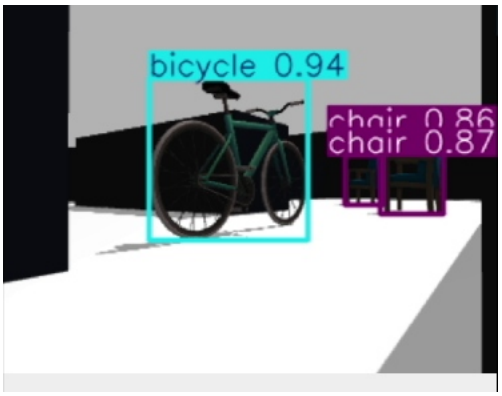
`record_demo.py` chỉ subscribe, không phát goal/vận tốc. Observer đối chiếu stamp/frame id ảnh annotated với RGB, đếm inference trong khi odometry có $|v_x| > 0.02$ m/s hoặc $|\omega_z| > 0.05$ rad/s; lưu action status, AMCL, ground truth, plan và hazard. Chỉ số này chứng minh xử lý ảnh khi có chuyển động theo ngưỡng, không tự chứng minh nhãn của mọi frame đều đúng.

10 Kết quả thực nghiệm và hình ảnh minh họa

10.1 Ảnh trích trực tiếp từ video demo đã cung cấp



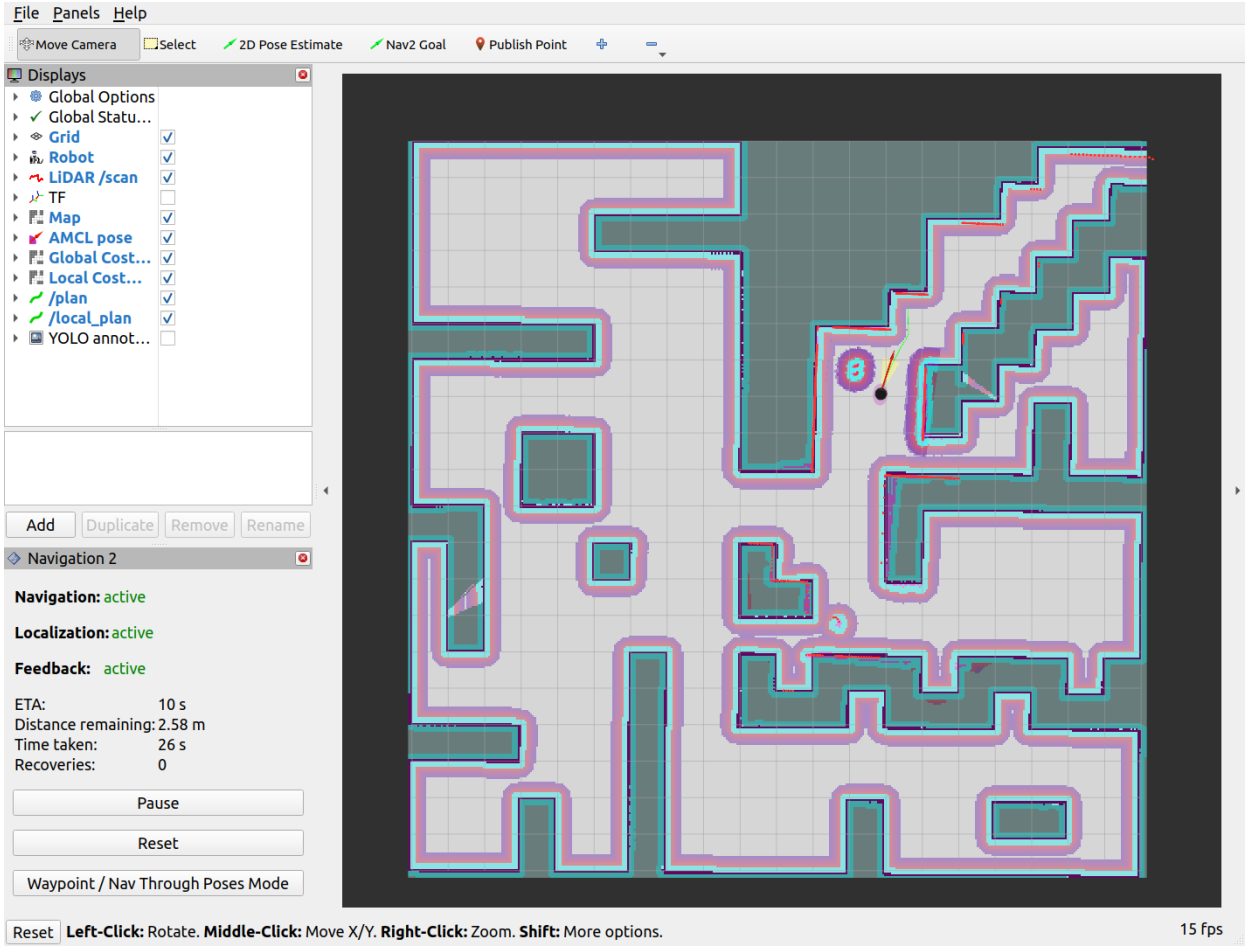
Hình 7: Khung hình tại 08:04.7 của video Drive: simulator có xe đạp và trụ cứu hỏa, RViz báo Navigation/Localization/Feedback active, còn 8.42 m và recovery 0; cửa sổ YOLO nhận bicycle 0.91 cùng lúc. Đây là ảnh trích video thật, không phải lần chạy mới do quá trình rà soát tạo ra.



Hình 8: Crop cửa sổ camera ở 15:49.9: bicycle 0.94 và chair 0.86/0.87. Crop chỉ bỏ phần desktop xung quanh; bbox và nhãn giữ nguyên từ video.

Video dài khoảng 16 phút 09 giây, độ phân giải 1920×1080, 60 FPS của bản ghi màn hình. FPS video không phải FPS camera hay inference. Thời điểm trên là vị trí trong tệp, không phải simulation time. Các khung trích xác nhận hình ảnh phiên bản có xe đạp và cửa sổ nhận diện; các bảng số liệu ở phần sau vẫn dùng log đúng lượt và phiên bản đã ghi.

10.2 Nạp map, localization và đường đi trên RViz2

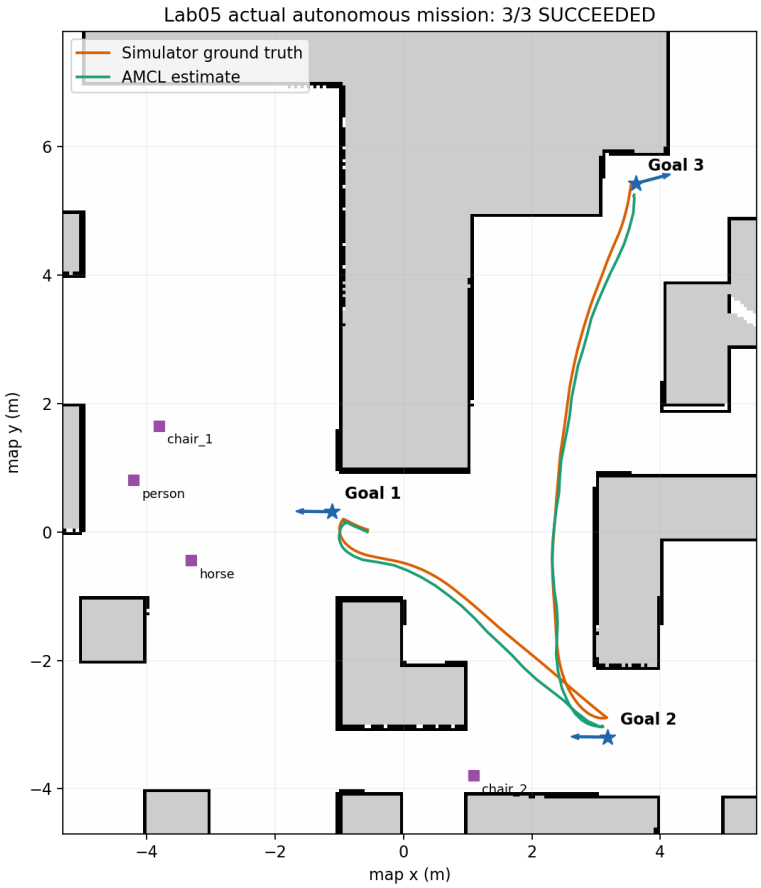


Hình 9: Ảnh RViz native của lượt đã lưu ngày 15/10/2026: Localization và Navigation active; scan, costmap, robot và path trong lúc Goal_3 đang chạy. Feedback còn 2.58 m, recovery 0 tại thời điểm chụp. Scene của ảnh thuộc phiên bản lịch sử sáu vật thể.

Ảnh cho thấy map đang được dùng với LiDAR và costmap; không phải map mới tạo bởi SLAM. Đường toàn cục là hướng di chuyển qua không gian có chi phí thấp, đường cục bộ là quỹ đạo ngắn từ DWB. Feedback trên ảnh là tiến độ tại thời điểm chụp, chưa phải chứng cứ SUCCEEDED; kết quả action sau đó được đối chiếu bằng UUID và log.

Nguồn ảnh: `validation/scattered_rviz_during_navigation.png`. Tệp `scattered_windows_during_navigation.json` ghi thời điểm native window đang hiển thị và trạng thái mission RUNNING; ảnh màn hình không được dựng lại bằng đồ họa để thay kết quả thực nghiệm.

10.3 Ba goal liên tiếp và quỹ đạo thực đo



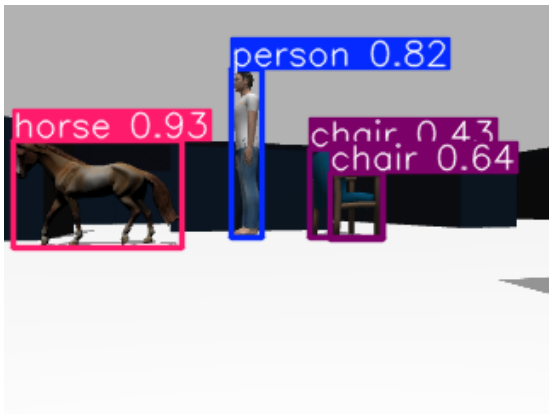
Hình 10: Quỹ đạo ground truth và AMCL của lượt baseline đã lưu ngày 15/10/2026. Sao và mũi tên chỉ goal/yaw; các marker tím là vật thể của scene bốn vật thể cũ, không phải bố trí tám vật thể hiện tại. Đây là quỹ đạo quan sát được, không phải đường nội suy giữa goal.

Bảng 12: Kết quả baseline và sai số vị trí gần thời điểm action kết thúc

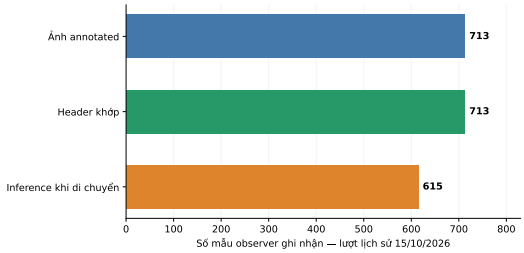
Goal	Status	Sai số AMCL (m)	Sai số truth (m)	$\lvert\Delta t\rvert$ (s)
Goal_1	SUCCEEDED (4)	0.246	0.211	0.138
Goal_2	SUCCEEDED (4)	0.174	0.302	0.143
Goal_3	SUCCEEDED (4)	0.178	0.073	0.081

Sai số được tính $e = \sqrt{(x - x_g)^2 + (y - y_g)^2}$ từ sample gần lúc action result, không phải phép đo chính xác tại thời điểm controller quyết định. Số liệu từ `demo_metrics.json`; chênh lệch truth/AMCL giải thích vì sao một action SUCCEEDED có thể đi kèm sai số truth lớn hơn xy tolerance.

10.4 Nhận diện thực tế và bằng chứng chạy đồng thời



Hình 11: Ảnh RGB annotated thực tế ở lượt baseline: person 0.82, horse 0.93 và hai bbox chair. Ảnh gốc 320×240, được phóng to trong báo cáo; đây là ảnh của phiên bản YOLO lịch sử, không phải phép đo YOLO11s mới.



Hình 12: Bằng chứng xử lý ảnh trong lượt sáu vật thể ngày 15/10/2026. Các số đếm lấy trực tiếp từ scattered_demo_summary.json.

Bảng 13: Tóm tắt lượt sáu vật thể đã lưu log

Chỉ số	Kết quả quan sát
Action/mission	3/3 status=4; completed=3; yolo_interrupted=false
RGB / annotated / header khớp	2561 / 713 / 713
Inference khi robot chuyển động	615; model thực ở cuda:0
Confidence lớn nhất đúng nhóm	person 0.925; horse 0.953; chair 0.895
Nhầm nhãn có trong log	dog 0.669; suitcase 0.624 (balô cũ); không tính là vật thể được nhận diện đúng
Hazard	Không ghi nhận bump, cliff, stall, wheel drop; có type 0 (backup limit)

Các confidence trong ảnh và bảng là từng prediction hoặc giá trị lớn nhất, không phải precision/recall/mAP. Lượt baseline riêng có median inference 116.13 ms, p95 156.55 ms, thời gian mission 162.34 s thực / 81.801 s mô phỏng (RTF khoảng 0.50). Các số này thuộc YOLO cũ; không dùng làm benchmark tốc độ YOLO11s với bộ lọc hiện tại.

11 Kiểm tra package và đánh giá kết quả

11.1 Kiểm tra mã nguồn ngày 20/10/2026

Bảng 14: Phạm vi và kết quả kiểm tra trong lần rà soát hiện tại

Hạng mục	Kết quả và phạm vi
Unit test	20/20 passed. Cấu hình/hash/vùng trống/quaternion, xác nhận bbox, geometry scene và shutdown navigation.
ROS timer/shutdown	Chạy test ở domain riêng 67; không gửi goal. Kiểm tra timeout khi clock mô phỏng đứng yên và giữ ưu tiên Ctrl+C.
Build	`colconbuild--symlink-install--packages-selectturtlebot4_lab05` thành công.
Launch/executable	Hai launch đọc được arguments; năm executable được đăng ký. Không khởi chạy simulator hay mission trong lần rà soát này.
Scene hiện tại	8/8 vật thể qua validator map/unknown/tường SDF/khoảng hở từng cặp. Hash map khớp dữ liệu đã lưu.
Bằng chứng runtime có sẵn	Đối chiếu JSON, UUID, ảnh RViz/YOLO, quỹ đạo và số liệu lịch sử; không gán thành benchmark phiên bản mới.

Đoạn mã 7: Lệnh tái lập kiểm tra không điều khiển robot

```
cd ~/turtlebot4_lab_ws
source lab_env.bash
source src/turtlebot4_lab05/scripts/lab05_env.bash
ROS_DOMAIN_ID=67 python -m pytest src/turtlebot4_lab05/test -q
colcon build --symlink-install --packages-select turtlebot4_lab05
python src/turtlebot4_lab05/scripts/check_scene.py
ros2 launch turtlebot4_lab05 bringup.launch.py --show-args
ros2 launch turtlebot4_lab05 yolo.launch.py --show-args
ros2 pkg executables turtlebot4_lab05
```

Các test dùng box tổng hợp kiểm tra logic xác nhận, không thay thế một dataset ảnh ở nhiều góc. Test scene kiểm tra mô hình hình học đã khai báo, không thay thế chạy robot trên bố trí mới. Test Ctrl+C mô phỏng state của client; không phải thử cắt mạng Nav2 thật.

11.2 Cách đối chiếu một mission thành công

Trong log `scattered_mission_events.json`, ba result có UUID khác nhau, status 4 và thứ tự Goal_1–Goal_2–Goal_3. Tóm tắt ghi completed=3 và yolo_interrupted=false. Observer đồng thời thấy 615 inference lúc odometry vượt ngưỡng chuyển động. Kết hợp ba loại bằng chứng mạnh hơn việc nhìn riêng ảnh terminal hoặc riêng ảnh camera.

Khả năng hoàn thành ba goal đã được kiểm chứng ở scene lịch sử. Sau khi thay balô bằng xe đạp, thêm vật thể và nâng model/bộ lọc, cần một lượt runtime mới nếu muốn kết luận cùng chỉ số cho cấu hình hiện tại. Video liên kết ở báo cáo là tài nguyên người thực hiện cung cấp; các bảng số liệu tự động vẫn chỉ dẫn tới các tệp log tương ứng.

12 Thảo luận, kiến thức rút ra và hướng phát triển

12.1 Những điểm cần hiểu khi giải thích chương trình

1. **Localization khác mapping:** mapping tạo/cập nhật map; AMCL sử dụng map có sẵn để ước lượng pose. Demo này nạp PGM cũ và không chạy SLAM.
2. **Goal khác vận tốc:** node Python gửi mục tiêu mức cao qua NavigateToPose. Nav2 lập đường, chấm quỹ đạo, phát vận tốc; Create 3/bridge thực thi. Có ba goal không có nghĩa viết ba lệnh cmd_vel cố định.
3. **Map frame khác odom frame:** map dùng định vị toàn cục; odom liên tục cục bộ. AMCL cung cấp hiệu chỉnh giữa chúng; pose/goal và TF phải nhất quán.
4. **Accepted khác succeeded:** server nhận goal chưa đồng nghĩa robot đã tới. Code chỉ tăng số mục tiêu hoàn thành khi action terminal status bằng 4.
5. **YOLO không thay costmap:** ảnh phát hiện người/ghế là thông tin thị giác 2D. LiDAR/-costmap mới là đầu vào tránh vật cản trong triển khai này.
6. **Đồng thời không đồng nghĩa cùng tần số:** camera có thể phát nhanh hơn inference. Latest-frame worker ưu tiên độ mới, bỏ frame cũ để tránh tích lũy trễ.
7. **Confidence không phải accuracy:** một nhãn confidence cao vẫn có thể sai trong domain mô phỏng. Cần dữ liệu có ground truth và metric theo lớp để định lượng độ chính xác.

12.2 Giới hạn và cách cải thiện có thể kiểm chứng

YOLO pretrained gặp domain gap khi mesh/texture, bóng và góc nhìn khác ảnh huấn luyện. Thay balô dễ nhầm bằng xe đạp làm thay đổi nội dung scene; không tự chứng minh mọi góc nhìn của xe đạp đều nhận đúng. Cần lấy nhiều frame ở các đoạn đường, gán nhãn, thống kê false positive/false negative rồi so sánh YOLO11n/11s trên cùng dữ liệu.

Bộ lọc hai frame giảm nhiễu thoáng qua nhưng có thể bỏ vật thể xuất hiện ngắn hoặc di chuyển nhanh trong ảnh. Hạ ngưỡng cây tăng recall tiềm năng đồng thời có thể tăng nhầm nhãn. Có thể tinh chỉnh từ tập validation hoặc fine-tune bằng ảnh mô phỏng đa góc, giữ tập đánh giá độc lập; không chọn vài confidence cực đại để báo cáo là độ chính xác tổng thể.

Muốn liên hệ vật thể với map cần depth hoặc hình học camera, TF camera–base–map, đồng bộ thời gian và ước lượng bất định. Chức năng đó chưa có trong bài hiện tại. Trên robot thật cũng cần xác minh lại sensor/topic, TF, tốc độ và đặc tính vật lý; kết quả maze mô phỏng không tự chuyển thành kiểm định thực địa.

12.3 Kết luận

Chương trình hoàn chỉnh kết nối map server–AMCL–Nav2 và một nhánh YOLO độc lập, có ba goal đo từ bài trước, quản lý trạng thái/action rõ ràng và lưu bằng chứng. Các log lịch sử xác nhận ba goal SUCCEEDED liên tiếp cùng inference khi robot chuyển động; bản hiện tại đã qua unit/build/geometry. Cấu trúc này cho phép tái lập demo, giải thích từng node/topic/action và tiếp tục đánh giá nhận diện có kiểm soát.

Tài liệu tham khảo và tài nguyên nộp bài

[1] Mã nguồn và cấu hình bài 05: Mã nguồn GitHub – assignments_2. Các nhận xét về state machine, thresholds, QoS, hash và xử lý ảnh được đối chiếu trực tiếp với source trong package.

[2] Video do người thực hiện cung cấp: Video demo – Google Drive. Liên kết ngắn trong PDF trỏ trực tiếp tới tệp Drive.

[3] Clearpath Robotics, TurtleBot 4 User Manual – Navigation. Tham khảo cấu trúc localization và navigation chính thức.

[4] Nav2, AMCL Configuration. Nguyên lý định vị và ý nghĩa tham số AMCL; giá trị trong báo cáo lấy từ bản Humble đang cài.

[5] Nav2, NavFn Planner Configuration và DWB Controller Configuration. Đối chiếu plugin NavFn/DWB với cấu hình tại máy.

[6] ROS 2 Humble, `nav2_msgs/action/NavigateToPose.action`, bản cài trong `/opt/ros/humble/share/nav2_msgs/action/`. Định nghĩa goal, feedback và trường result; status kết thúc do ROS action wrapper cung cấp.

[7] Ultralytics, YOLO11 và Predict mode. Cấu hình model.predict và trường bbox của Results; phiên bản thư viện sử dụng là 8.3.221.

[8] Bằng chứng nội bộ: `validation/demo_metrics.json`, `scattered_demo_summary.json`, `scattered_mission_events.json`, `scene_layout.json`; ảnh native trong thư mục validation. Các số liệu có cùng nguồn được mô tả trong chú thích bảng/hình.

Tái lập báo cáo: mã LaTeX và ảnh nằm trong `report/` của package. Có thể biên dịch bằng pdfLaTeX hai lần hoặc Tectonic. Các sơ đồ pipeline là TikZ vector; sơ đồ map và biểu đồ số liệu được dựng bằng script từ map/config/JSON. Ảnh RViz/camera giữ nguồn thực nghiệm và được chú thích phiên bản.